

Software Test Plan (STP)

Problem Statement No. 15 — Railway Reservation System

Project: Railway Reservation System (RRS)

Version: 1.0

Authors: Syed Obaid Faraz (PES2UG24CS546) · Simon C A (PES2UG24CS503) · Sudha (PES2UG24CS529) · Tanisha Khorla (PES2UG24CS551) — Group 5

Date: 29-09-2026

Status: Draft — Submitted for Review

1. Introduction

Purpose: This document defines the test plan for the Railway Reservation System (RRS) v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing the core booking flow described in the project SRS.

Scope: Testing covers passenger registration and login, train search, ticket booking and payment, ticket cancellation, PNR status enquiry and booking history, and admin management of trains. Multi-quota fare logic, Tatkal booking, automatic waitlist/RAC promotion, and large-scale load testing are out of scope, consistent with the SRS scope.

References: RRS Software Requirements Specification (SRS) v1.0, Problem Statement No. 15.

Definitions: PNR (Passenger Name Record), UI (User Interface), API (Application Programming Interface), JWT (JSON Web Token), HTTPS (Hypertext Transfer Protocol Secure), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), STP (Software Test Plan).

2. Test Items

- Authentication module (registration, login, logout, password reset)
- Train search & seat availability module
- Ticket booking & payment module
- Ticket cancellation module
- PNR status & booking history module
- Admin train-management module

3. Features to be Tested

Features mapped to SRS requirement IDs:

- RRS-F-001-004: Registration, login, logout, password reset
- RRS-F-010-012: Train search, seat availability, fare display
- RRS-F-020-023: Booking, payment, PNR generation
- RRS-F-030-032: Cancellation and seat release
- RRS-F-040-041: PNR status enquiry, booking history
- RRS-F-050-052: Admin train management and bookings view
- RRS-NF-001-005: Performance, usability, reliability, availability, maintainability
- RRS-SR-001-005: Security requirements (password hashing, session auth, input validation, RBAC, HTTPS)

4. Features Not to be Tested

- Multi-quota fare/concession logic and Tatkal booking (out of SRS scope)
- Automatic waitlist/RAC promotion (out of SRS scope)
- Payment gateway's own internal processing (assumed tested by the provider; only the integration point is tested)
- Large-scale concurrent load beyond a small classroom-demo simulation

5. Test Approach / Strategy

Levels

- Unit tests (module-level, e.g., fare calculation, seat-count logic)
- Integration tests (Booking module ↔ Payment step ↔ Database)
- System tests (end-to-end booking flow through the UI)
- Acceptance tests (UAT against SRS acceptance criteria)

Types

- Functional testing (core features)
- Regression testing (after each feature-branch merge)
- Basic performance testing (search/booking response time)
- Usability testing (first-time-user walkthrough)

Entry Criteria: Stable build delivered on the integration branch, test data available, test environment ready.

Exit Criteria: 100% of planned test cases executed, 0 critical defects open, all acceptance criteria in the SRS satisfied.

5.1 Security Validation

- Verify passwords are stored hashed, never logged in plain text
- Verify HTTPS is enforced on all connections
- Verify session tokens are required for booking/cancellation/admin actions and rejected when invalid or expired
- Basic input-validation checks on booking and admin forms (SQL injection / XSS attempts)
- Verify admin-only actions are blocked for a non-admin logged-in user

6. Test Environment

Hardware: Standard developer laptops/desktops; no special hardware required.

Software: RRS web application (frontend + backend), test database instance, browser (Chrome/Firefox).

Tools: Postman (API testing), Jest / a similar unit-test framework (backend unit tests), Jira (test-case tracking and defect logging), Git/GitHub (version control and branch-based test builds).

Test Data: Seeded dummy user accounts, sample trains with limited seat counts (to test boundary/over-booking cases), and test bookings.

7. Test Schedule

Milestones:

- Test case design: aligned with each module's feature-branch creation in Jira
- Environment setup: once the initial backend + database scaffold is merged to the integration branch
- Test execution: ongoing, alongside each feature-branch pull request
- Final regression pass: before the final submission/demo milestone
- UAT / demo walkthrough: final week before submission

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual)
- Test Data
- Test Execution Logs
- Defect Reports (as Jira issues)
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Tanisha Khoria	Prepare test plan, coordinate execution, sign off on test summary
Test Engineer	Sudha	Design & execute test cases, log defects in Jira
Developer	Syed Obaid Faraz	Support defect fixes and triage
Developer	Simon C A	Support defect fixes and triage
Product Owner	Group 5 (rotating)	Approve test results, sign off on readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in a stable build reaching the integration branch	Test each feature branch independently before merge; request early builds from the module owner
Local/test environment inconsistency across team members' machines	Use a shared setup script / README and a common test-database seed
Limited time for full regression before submission	Prioritise high-priority requirements (per SRS priority column) for full regression; keep the feature set small

11. Assumptions & Dependencies

- The backend API and database schema are stable before system testing begins
- Test data (dummy accounts, trains) will be seeded before execution
- All team members have access to the shared Git repository and Jira board

12. Suspension & Resumption Criteria

Suspend testing if:

- The shared test environment is unavailable for more than 4 hours

- The build is too unstable to test (blocks more than 30% of planned test cases)

Resume testing if:

- Blocking defects are resolved and merged
- The environment is stabilised

13. Test Case Management & Traceability

The RTM in the SRS document maps each requirement to its test case ID(s); the same IDs are used here and tracked as Jira issues/sub-tasks. Examples:

- RRS-F-001 (Registration) → TC-Reg-01
- RRS-F-020 (Booking) → TC-Book-01, TC-Book-02
- RRS-F-023 (PNR generation) → TC-Pay-02
- RRS-NF-003 (No over-booking) → TC-Integrity-01

14. Test Metrics & Reporting

Metrics collected

- % test cases executed
- % passed / failed
- Defect count by severity
- Requirement coverage (FRs/NFRs with at least one passing test case)

Reports

- Test execution status, tracked on the Jira board
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead		
Dev Lead		
Product Owner		
Course Coordinator		